

Visual Recognition

Mini Project PART 1

Project Title: Multi-Object Apparel Detection and Instance Segmentation

1. Problem Motivation and Overview

With the rapid growth of fashion and e-commerce platforms, there is a strong need for automated visual recognition systems that can accurately identify and analyze clothing items in images. Such systems play a crucial role in applications like product catalog generation, visual search, inventory management, and fashion analytics.

The objective of this mini project is to design a visual recognition system capable of handling multiple clothing items within a single image. The system will address three core tasks: ✓classification, where an image is assigned to a specific product category; ✓detection, which involves locating clothing items by predicting their bounding boxes; and ✓segmentation, where pixel-level masks are generated to precisely segment each clothing item instance.

2. Constraints

- **Compute:** All training and experiments must be feasible on Kaggle or Google Colab.
- **Data Scale:** There is no restriction on the number of samples used from the provided dataset; however, students must choose the data size carefully based on the available compute resources.
- **Model Size:** The chosen models should be able to run within the given compute limits. Extremely large models are not permitted; as a guideline, models should have fewer than 7B parameters.
- **Course Scope:** Only model architectures covered in the syllabus may be used. Transfer learning and fine-tuning are allowed and encouraged.

3. Dataset Description

Here is a clothing dataset containing **person-centric images**. The dataset provides **item-level annotations** for the clothing items present in each image, and a single image may contain **multiple clothing items**.

Dataset link: [link to dataset](#).

Password: 2019Deepfashion2**

Each image in a separate image set has a unique six-digit number such as 000001.jpg. A corresponding annotation file in json format is provided in annotation set such as 000001.json.

Each annotation file is organized as below:

- **source:** a string, where 'shop' indicates that the image is from a commercial store while 'user' indicates that the image is taken by users.

- **pair_id**: a number. Images from the same shop and their corresponding consumer-taken images have the same pair id.
 - **item 1**
 - **category_name**: a string which indicates the category of the item.
 - **category_id**: a number which corresponds to the category name. In **category_id**, 1 represents short sleeve top, 2 represents long sleeve top, 3 represents short sleeve outwear, 4 represents long sleeve outwear, 5 represents vest, 6 represents sling, 7 represents shorts, 8 represents trousers, 9 represents skirt, 10 represents short sleeve dress, 11 represents long sleeve dress, 12 represents vest dress and 13 represents sling dress.
 - **style**: a number to distinguish between clothing items from images with the same pair id. Clothing items with different style numbers from images with the same pair id have different styles such as color, printing, and logo. In this way, a clothing item from shop images and a clothing item from user image are positive commercial-consumer pair if they have the same style number greater than 0 and they are from images with the same pair id.(If you are confused with style, please refer to issue#10.)
 - **bounding_box**: [x1,y1,x2,y2], where x1 and y_1 represent the upper left point coordinate of bounding box, x_2 and y_2 represent the lower right point coordinate of bounding box. (width=x2-x1;height=y2-y1)
 - **landmarks**: [x1,y1,v1,...,xn,yn,vn], where v represents the visibility: v=2 visible; v=1 occlusion; v=0 not labeled. We have different definitions of landmarks for different categories. The orders of landmark annotations are listed in figure 2.
 - **segmentation**: [[x1,y1,...xn,yn],[]], where [x1,y1,xn,yn] represents a polygon and a single clothing item may contain more than one polygon.
 - **scale**: a number, where 1 represents small scale, 2 represents modest scale and 3 represents large scale.
 - **occlusion**: a number, where 1 represents slight occlusion(including no occlusion), 2 represents medium occlusion and 3 represents heavy occlusion.
 - **zoom_in**: a number, where 1 represents no zoom-in, 2 represents medium zoom-in and 3 represents large zoom-in.
 - **viewpoint**: a number, where 1 represents no wear, 2 represents frontal viewpoint and 3 represents side or back viewpoint.
 - item 2
 - ...
 - item n

3.1 Data Preprocessing

- Prune the dataset by selecting the **top 5 most frequently occurring clothing categories**. Evaluation will be performed only on these five categories.

- Parse the annotation file for each image and extract the relevant information required for **classification, detection, and segmentation**.
- Handle **class imbalance** using appropriate techniques such as class reweighting, resampling, or data augmentation.
- Create standard **training, validation, and test splits** for model development and evaluation.

3. Task Definition

3.1 Classification:

Given an input image, identify all clothing categories present anywhere in the image. This is a **multi-label classification** task, where a CNN is used to predict all clothing classes appearing in the image without performing any localization.

3.2 Detection and Segmentation

Given an input image, detect and classify all clothing items and also determine their precise locations. More formally:

- **Input:** An image containing one or more persons wearing clothing items.
- **Output:** A set of bounding boxes, corresponding clothing category labels, and segmentation masks for each detected clothing instance.

4. Architectures

4.1 Architectures for the Classification Task

For **Task 3.1 (multi-label image-level clothing classification)**, you must choose **any one** of the following CNN architectures and adapt it for multi-label classification using **sigmoid activation** and **binary cross-entropy loss**:

- **ResNet-50**
- **EfficientNet-B0** or ~~EfficientNet-B2~~ from scratch train
transfer learning
- **MobileNetV3** TODO

4.1 Architectures for Detection and Segmentation Task

For **Task 3.2 (detection and segmentation)**, the following architectures perform **bounding box prediction, category classification, and segmentation jointly**. You are required to **train all three models**.

- **YOLO**

Backbone: CNN-based feature extractor (e.g., CSPDarknet or YOLOv8 backbone) for hierarchical feature extraction.

Neck: Feature aggregation module (FPN/PAN) to combine multi-scale features for detecting clothing items of varying sizes.

Heads:

- a. Detection head: Predicts bounding box coordinates, objectness scores, and clothing category probabilities at multiple scales.
- b. Segmentation head: Predicts instance-level segmentation masks using learned mask coefficients and prototype masks.

- **Mask R-CNN**

Backbone: ResNet-50 or ResNet-101 for feature extraction.

Feature Pyramid Network (FPN): Generates multi-scale feature representations.

Region Proposal Network (RPN): Proposes candidate regions likely to contain clothing items.

ROI Heads:

- a. Classification head: Predicts the clothing category for each region of interest.
- b. Bounding box regression head: Refines object localization.
- c. Mask head: Produces pixel-level binary masks for each detected instance.

- **U-Net**

Encoder: CNN-based encoder that progressively downsamples the input to extract high-level semantic features. The encoder may be trained from scratch or initialized with a pretrained backbone (e.g., ResNet-34).

Decoder: Symmetric upsampling path with skip connections to recover spatial details while preserving semantic information.

Output: Generates pixel-wise segmentation maps for clothing regions across categories.

Post-processing:

- a. Connected component analysis is applied to separate individual clothing instances.
- b. Bounding boxes are extracted from each connected component.
- c. Instance-level category labels are assigned using majority voting or region-based classification.

5. Training Methodology

For both the tasks, all models must be trained using the **same training split** and evaluated with **a common evaluation protocol** to enable fair comparison. You will follow two different training strategies:

- **Training from scratch:** Train the model end-to-end with randomly initialized weights to establish a reference baseline (performance may be limited).
- **Transfer learning:** Initialize the model with pretrained weights, freeze the initial layers, and train only the final layers to improve performance and convergence.

6. Evaluation metric

- For multi-label image-level clothing classification, performance will be evaluated using **per-class Precision, Recall, and F1-score**. In addition, **macro-averaged and micro-averaged F1-scores** must be reported to capture both class-balanced performance and overall prediction quality. **ROC curves and Area Under the Curve (AUC)** should also be reported per class to analyze threshold-independent classification performance.
- Segmentation performance will be evaluated using **Mean Intersection over Union (mIoU)**, computed per clothing category and macro-averaged across the selected five classes to assess mask overlap quality. In addition, the **Dice coefficient (F1-score for segmentation masks)** must be reported to analyze performance under class imbalance conditions.
- Detection performance will be measured using **COCO-style mean Average Precision (mAP@[0.5:0.95])**, which evaluates detection and localization accuracy across multiple IoU thresholds. To enable detailed per-class analysis and better understanding of false positives and false negatives, **ROC curves, AUC, and F1-score** must also be reported.

7. Hidden set evaluation

A **hidden test set evaluation** will be conducted using an automated inference pipeline.

Each team is required to:

- Upload their **best-performing model for Tasks 3.1 and 3.2**, chosen from the implemented architectures, to a **public Hugging Face repository**.
- Ensure the uploaded model adheres to the specified **compute and model size constraints**.

The evaluation procedure is as follows:

- Our inference script will automatically download the submitted model.
- The script will run inference on a **private hidden test set**.
- Final scores will be computed using the evaluation metrics described in **Section 6**.
- The script will also verify:
 - **Correct output formatting**
 - **Successful completion of inference within the prescribed resource limits**

Only models that pass all validation checks will be considered for final grading.

8. Deliverables

- **Data Loader and Preprocessing Code:** Python code for dataset loading and preprocessing, including label mapping, train/validation/test splits, and required image transformations.
- **Training Code:** Separate training notebooks or scripts for each architecture used in the project, including CNN-based classifiers, YOLO, Mask R-CNN, and U-Net.
- **Results and Analysis:** A comparative performance table reporting all evaluation metrics described in Section 6, along with qualitative visualizations. This section must include key observations, discussion, and conclusions drawn from the experiments.
- **Model Release:** A public Hugging Face repository containing the best-performing model for Tasks 3.1 and 3.2, including trained weights and clear inference instructions.
- **Final Report:** A comprehensive PDF report detailing design choices, model architectures, training methodology, experiments conducted, evaluation results, and overall findings.

9. Evaluation Rubrics

Component	Points	Expectation
Viva (Oral)	5	Clear explanation of dataset preprocessing, model choices, and evaluation methodology; ability to justify design decisions and interpret failure cases.
Inference Script Submission (Hugging Face)	10	Sub-Component Breakdown: • Reproducibility & Integration (3 pts): Model is correctly uploaded to a public Hugging Face repository with clear inference instructions. The script runs seamlessly with the automated inference system without manual intervention. • Output Validity (3 pts): Model produces outputs in the exact required format: bounding boxes, category labels, and segmentation masks. • Quantitative Performance (4 pts): Achievement of high scores in required metrics: macro-averaged and micro-averaged Precision, Recall, and F1-scores for classification, mIoU and Dice for segmentation, and mAP@[0.5:0.95] for detection. Scoring for this section will be calculated relative to the performance of the rest of the batch.
Report (PDF)	5	Expected Sections & Content: • Dataset & Preprocessing • Experimental Details • Comparative Analysis • Discussion & Qualitative Examples